

Deep Reinforcement Learning

Deep Q Learning

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- On-policy control with gradients and semi-gradients
 - Introduction & extension of existing algorithms
 - Gradient MC
 - Semi-gradient SARSA
 - Challenges (Levine: Data correlation & semi-gradient)
- Deep Q networks
 - Replay buffers
 - Target networks & alternative target networks (Polyak)
 - Double Q networks
 - Prioritized experience replay
 - n -step returns
 - Some remarks on continuous actions

- Least squares policy iteration

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q-learning	Q learning with neural networks
9	Policy gradients	
10	Actor-critic algorithms	
11	Advanced algorithms	
	Model-Based Control	
	Advanced Topics	

Recap: Value function approximation

- We want to **approximate the value function by parametric model** with trainable parameters $\theta \in \mathbb{R}^d$, i.e., $V_\theta(s) \approx V^\pi(s)$.
- For training, we need to define a **prediction objective**:

$$J(\theta) = \sum_{k=1}^N \left(V^\pi(s_k) - V_\theta(s_k) \right)^2 \approx \int_{\mathcal{S}} \mu(s) \left(V^\pi(s) - V_\theta(s) \right)^2 ds = \overline{VE}(\theta). \quad (1)$$

- **Incremental weight updates** via
 - gradients, e.g., in the Monte Carlos setting:

$$\theta \leftarrow \theta + \alpha [g - V_\theta(s)] \nabla_\theta V_\theta(s). \quad (2)$$

- semi-gradients in the bootstrapping case: we are not differentiating the target, which also depends on V_θ . For TD learning, we have

$$\theta \leftarrow \theta + \alpha [r + \gamma V_\theta(s') - V_\theta(s)] \nabla_\theta V_\theta(s). \quad (3)$$

- **Batch updates** using $\mathcal{B} = \{(s_i, r_i, s'_i)\}_{i=1}^b \subset \mathcal{D}$ instead of a single sample as in SGD. For TD updates, we thus get

$$\theta \leftarrow \theta + \frac{\alpha}{b} \sum_{(s_i, r_i, s'_i) \in \mathcal{B}} [r_i + \gamma V_\theta(s'_i) - V_\theta(s_i)] \nabla_\theta V_\theta(s_i). \quad (4)$$

- For **linear models** (i.e., $V_\theta(s) = \theta^\top \psi(s) = \sum_{k=1}^d \theta_k \psi_k(s)$), training can be done in one step via linear regression:

$$\theta^* = (\Psi^\top \Psi)^{-1} \Psi^\top y = \Psi^\dagger y \quad \Rightarrow \quad V_{\theta^*}(s) = \theta^{*\top} \psi(s) = \sum_{k=1}^d \theta_k^* \psi_k(s). \quad (5)$$

On-policy control with gradients and semi-gradients

Value-based control: Q function approximation

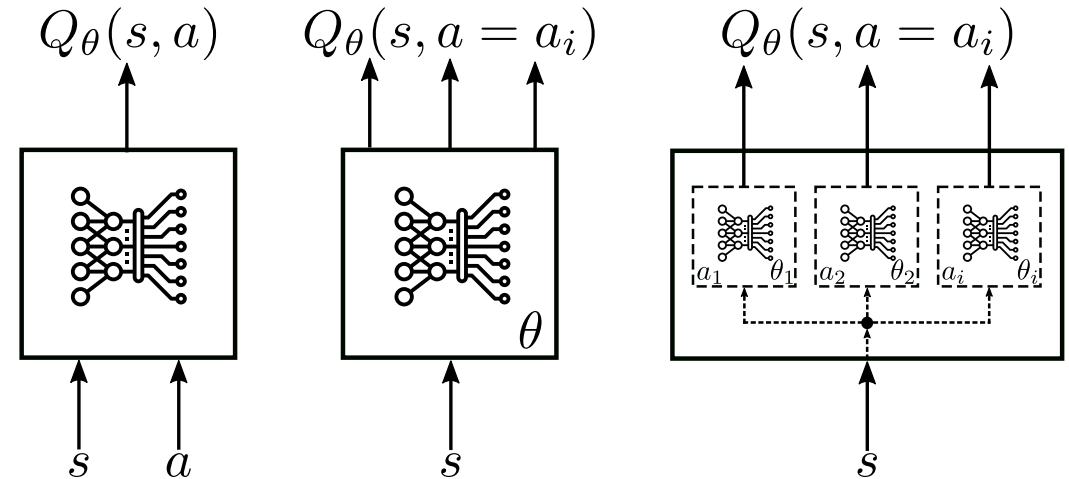
Today's focus: **value-based control** $\Rightarrow$ How to transfer tabular methods to the function approximation case.

- The states are continuous, but the actions are (usually) still discrete.
- Central object of interest: The Q function that we wish to approximate with a parametric model (e.g., a neural net)

$$Q_{\theta}(s, a) \approx Q^{\pi}(s, a)$$

- We can then use this in the well-known generalized policy iteration (GPI) scheme to identify optimal policies:

$$Q_{\theta}(s, a) \approx Q^*(s, a) \quad \Rightarrow \quad \pi^*(s) = \arg \max_{a \in \mathcal{A}} Q_{\theta}(s, a).$$



Types of architectures (Abdelwanis et al. 2026)

Gradient-based action value learning

The transfer from value function approximation is straightforward!

- Adaptation of (1) to get $J(\theta)$

$$J(\theta) = \sum_{k=1}^N \left(Q^\pi(s_k, a_k) - Q_\theta(s_k, a_k) \right)^2. \quad (6)$$

- Incremental update using (semi-)gradients:

$$\theta \leftarrow \theta + \alpha [Q^\pi(s, a) - Q_\theta(s, a)] \nabla_\theta Q_\theta(s, a). \quad (7)$$

- Depending on the control approach, the true target $Q^\pi(s, a)$ is approximated by:
 - Monte Carlo: full episodic return

$$Q^\pi(s, a) \approx g,$$

- SARSA: one-step bootstrapped estimate

$$Q^\pi(s, a) \approx r + \gamma Q_\theta(s', a'),$$

Algorithm: Gradient MC control

Algorithm: Gradient Monte Carlo algorithm for estimating Q^π / Q^*

Input:

- a differentiable, parameter-dependent function $Q_\theta : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$
- learning rate α
- **Prediction case:** a policy π
- **Improvement case:** ϵ defining the ϵ -greedy policy update based on Q_θ

Initialize: Value function weights $\theta \in \mathbb{R}^d$ arbitrarily

for $k = 1, 2, \dots, K$ episodes:

Generate a sequence following π (or ϵ -greedy(Q_θ)):

$$((s_0, a_0, r_0), (s_1, a_1, r_1), \dots, (s_{T_k-1}, a_{T_k-1}, r_{T_k-1}))$$

Calculate the every-visit returns g_t

for $t = 0, 1, \dots, T - 1$:

$$\theta \leftarrow \theta + \alpha [g_t - Q_\theta(s_t, a_t)] \nabla_\theta Q_\theta(s_t, a_t)$$

- Direct transfer from tabular case to function approximation.
- Update target becomes the sampled return $Q^\pi(s_t, a_t) \approx g_t$.
- If operating ϵ -greedy on Q_θ : baseline policy (given by $\theta^{(0)}$) must terminate the episode!

Algorithm: Semi-gradient SARSA

Algorithm: Semi-gradient SARSA for estimating Q^π / Q^*

Input:

- a differentiable, parameter-dependent function $Q_\theta : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$
- learning rate α
- **Prediction case:** a policy π
- **Improvement case:** ϵ defining the ϵ -greedy policy update based on Q_θ

Initialize: Value function weights $\theta \in \mathbb{R}^d$ arbitrarily

for $k = 1, 2, \dots, K$ episodes:

Initialize s_0

for $t = 0, 1, \dots, T - 1$:

Obtain $a_t \sim \pi(\cdot | s_t)$ (or $a_t \sim \epsilon$ -greedy($Q_\theta(s_t, \cdot)$))

Observe r_t and s_{t+1}

if s_{t+1} is terminal then

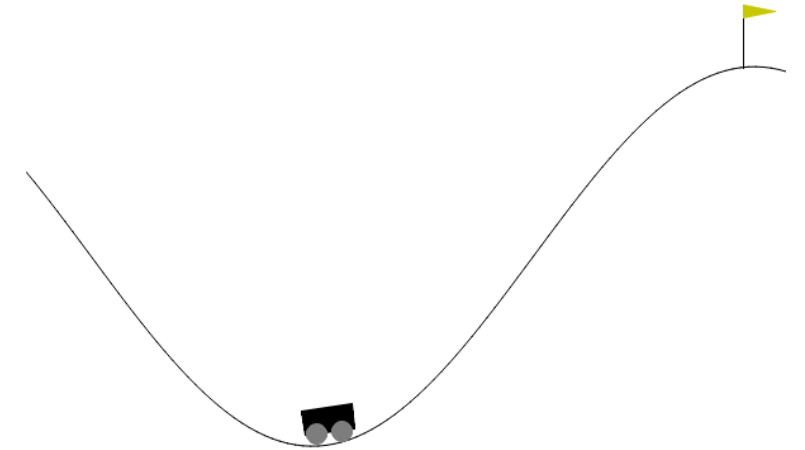
$\theta \leftarrow \theta + \alpha[r_t - Q_\theta(s_t, a_t)] \nabla_\theta Q_\theta(s_t, a_t)$

Go to next episode $k + 1$

Choose $a' \sim \pi(\cdot|s_{t+1})$ (or $a' \sim \epsilon\text{-greedy}(Q_\theta(s_{t+1}, \cdot))$)
 $\theta \leftarrow \theta + \alpha[r_t + \gamma Q_\theta(s_{t+1}, a') - Q_\theta(s_t, a_t)] \nabla_\theta Q_\theta(s_t, a_t)$

Example: Mountain car (1)

- Continuous state $s \in \mathcal{S} = [x_{\min}, x_{\max}] \times [v_{\min}, v_{\max}]$ (minimal and maximal position x and velocity v).
- Single, discrete action $a \in \mathcal{A} = \{-1, 0, 1\}$.
- $r = -1$ (goal is to terminate episode as quick as possible).
- Episode terminates when car reaches the flag (or max steps).
- Simplified longitudinal car physics with state constraints.
- Position initialized randomly within valley, zero initial velocity.
- Car is underpowered and requires swing-up.

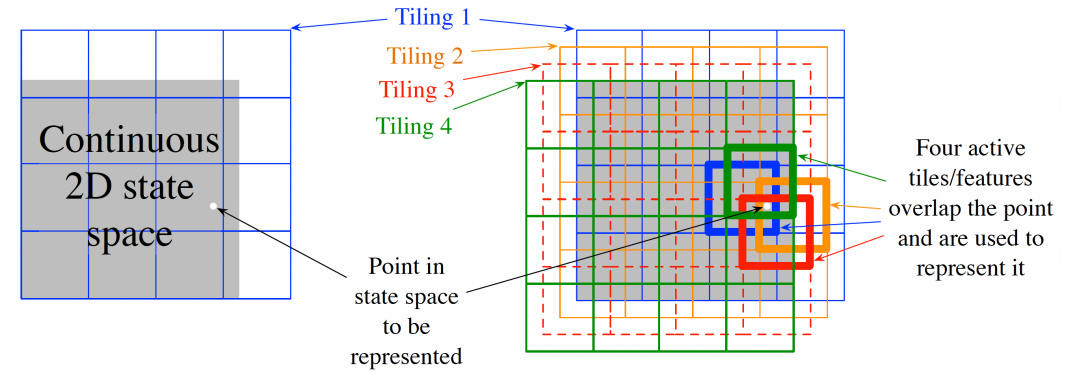


Mountain car (Sutton and Barto 1998, Gymnasium)

Approximation: Linear model with *tile coding* features $x_i(s, a)$, where

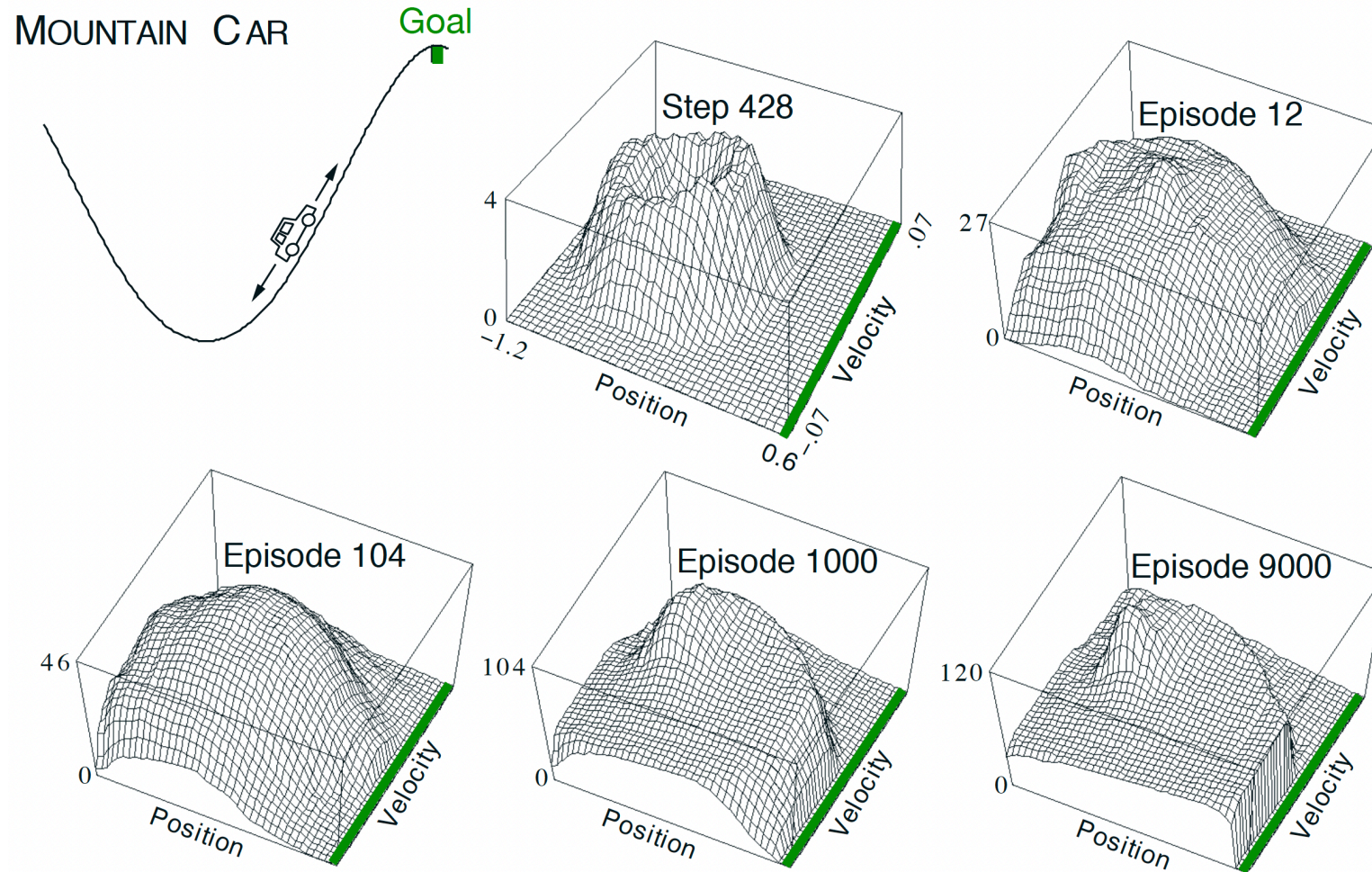
$$\psi(s, a) = \sum_{i=1}^d \theta_i x_i(s, a),$$

$$x_i(s, a) = \begin{cases} 1 & (s, a) \in \text{tile } \#i \\ 0 & \text{otherwise} \end{cases}.$$



Tile coding (Sutton and Barto 1998, Figure 9.9)

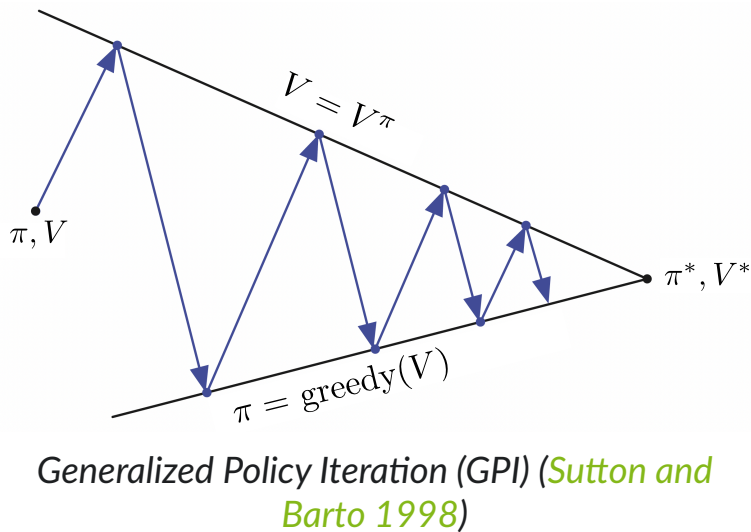
Example: Mountain car (2)



The Mountain Car task (upper left panel) and the cost-to-go function ($-\max_{a \in \mathcal{A}} Q_{\theta}(s, a,)$) learned during one run. (Sutton and Barto 1998, Figur 10.1)

Warning: a huge drawback of function approximation

- Recall tabular **policy improvement theorem**: guarantee to find a globally better or equally good policy in each update step.
- Parameter updates of the form (7) have an impact on $Q(s, a)$ for **all** s and a , not just the one we visited!



Loss of the policy improvement theorem

- When using function approximation, the policy improvement theorem does no longer apply.
- While improving the policy, we may impair it in other places *at the same time*.

Deep Q networks (DQN)

General strategy behind DQN

- Introduced in the famous DeepMind paper ([Mnih et al. 2015](#)).
- Same off-policy strategy as in the *classical/tabular* Q learning algorithm:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a' \in \mathcal{A}} Q(s', a') - Q(s, a) \right].$$

- Now: incremental learning using semi-gradients (Eq. (7)) and TD(0) bootstrapping:

$$\theta \leftarrow \theta + \alpha \left[r + \gamma \max_{a' \in \mathcal{A}} Q_{\theta}(s', a') - Q_{\theta}(s, a) \right] \nabla_{\theta} Q_{\theta}(s, a). \quad (8)$$

But with two important distinctions

1. An **experience replay buffer** for batch learning replaces the single-sample updates.

The motivation behind this

1. **Sample efficiency**
 - *Experience replay*: efficiently use available data.
 - *Stabilize learning* by reducing the correlation between samples
 $\Rightarrow$ sampling from a replay buffer leads to i.i.d.-like data.
2. **Stabilization of learning**

2. A **separate set of weights** θ' for the bootstrapped Q-target ((8) is **not** a gradient).

- targets don't change in inner loop $\Rightarrow$ well-defined learning problem.

DQN working principle (1)

- Take actions u based on $Q_\theta(s, a)$ (e.g., ϵ -greedy).
- Store observed tuples (s, a, r, s') in memory buffer $\mathcal{D}$.
- Sample mini-batches $\mathcal{B}$ from $\mathcal{D}$.
- Calculate bootstrapped Q-target with a target network with weights θ' :

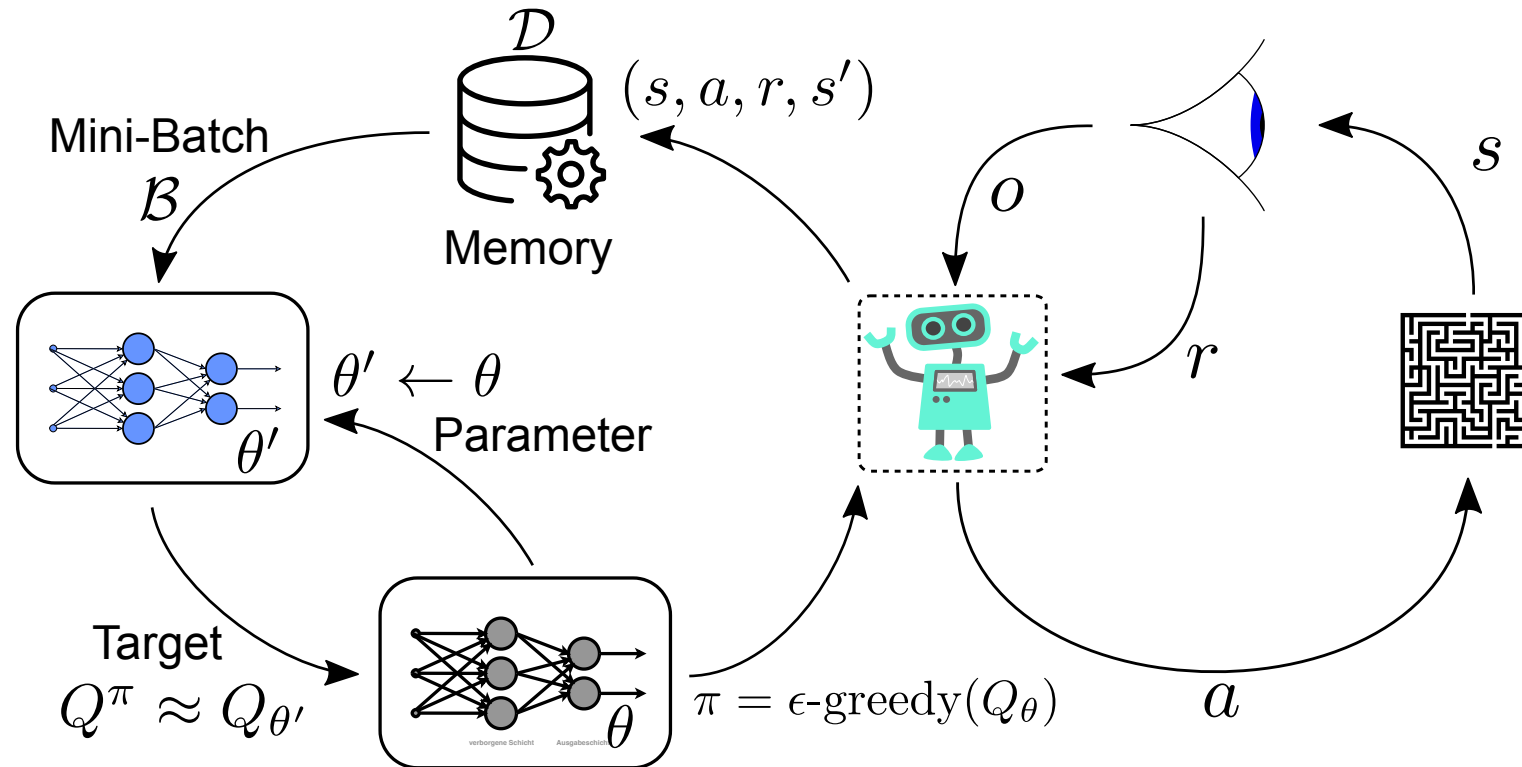
$$Q^\pi(s, a) \approx r + \max_{a \in \mathcal{A}} Q_{\theta'}(s, a).$$

- Optimize MSE loss between above targets and the regular approximation $Q_\theta(s, a)$ using $(s_i, a_i, r_i, s'_i) \in \mathcal{B}$:

$$L(\theta) = \sum_{i=1}^{|\mathcal{B}|} \left(\underbrace{r_i + \max_{a \in \mathcal{A}} Q_{\theta'}(s'_i, a)}_{=y_i \text{ (target)}} - Q_\theta(s_i, a_i) \right)^2,$$
$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{|\mathcal{B}|} [y_i - Q_\theta(s_i, a_i)] \nabla_\theta Q_\theta(s_i, a_i).$$

- Update θ' based on θ from time to time.

DQN working principle (2)



Adapted from (Abdelwanis et al. 2026)

The DQN algorithm

Algorithm: The “classic” DQN algorithm (Mnih et al. 2015)

Input:

- a differentiable, parameter-dependent function $Q_\theta : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$
- learning rate α , greedy parameter ϵ , update rate $N_\theta \in \mathbb{N}$

Initialize: Q function weights $\theta = \theta' \in \mathbb{R}^d$ arbitrarily, $\mathcal{D} = \{\}$

for $k = 1, 2, \dots, K$ episodes:

Initialize s_0

for $t = 0, 1, \dots, T - 1$:

Obtain $a_t \sim \epsilon$ -greedy($Q_\theta(s_t, \cdot)$) and observe r_t and s_{t+1}

Store tuple (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$

Sample minibatch $\mathcal{B}$ from $\mathcal{D}$ (after initial warm-up)

Compute targets for $(s_i, a_i, r_i, s'_i) \in \mathcal{B}$

$$y_i = r_i + \max_{a \in \mathcal{A}} Q_{\theta'}(s'_i, a) \quad (\text{or } y_i = r_i \text{ if } s'_i \text{ is terminal})$$

Update θ via gradient descent on $L(\theta)$ with learning rate α
if $t \bmod N_\theta = 0$ then $\theta' \leftarrow \theta$

Remarks:

1. For the standard MSE loss

$$L(\theta) = \sum_{i=1}^{|\mathcal{B}|} \left(y_i - Q_\theta(s_i, a_i) \right)^2,$$

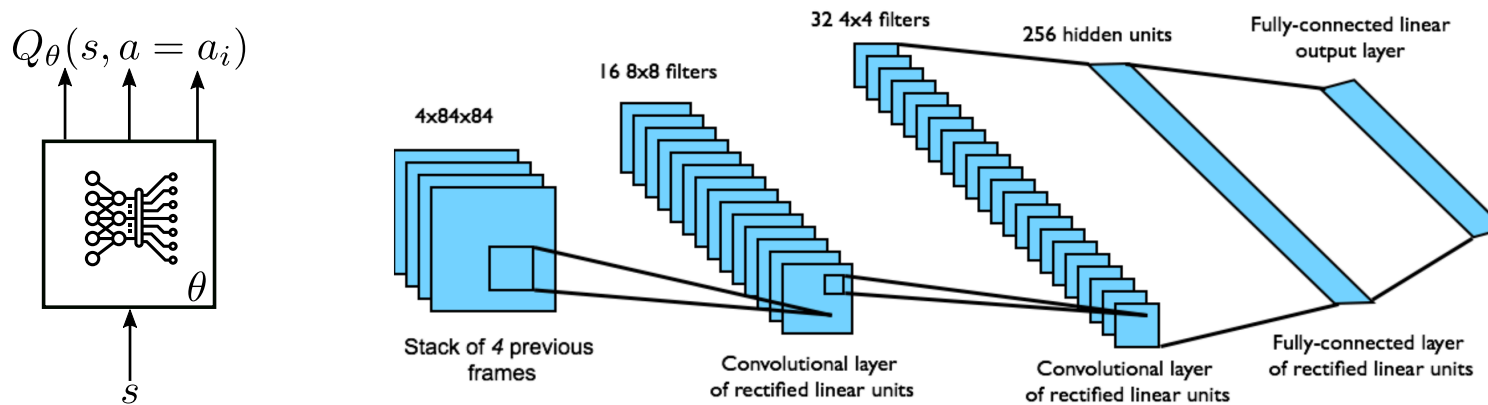
the weight update becomes the one we have seen before:

$$\theta \leftarrow \theta + \alpha \sum_{i=1}^{|\mathcal{B}|} [y_i - Q_\theta(s_i, a_i)] \nabla_\theta Q_\theta(s_i, a_i).$$

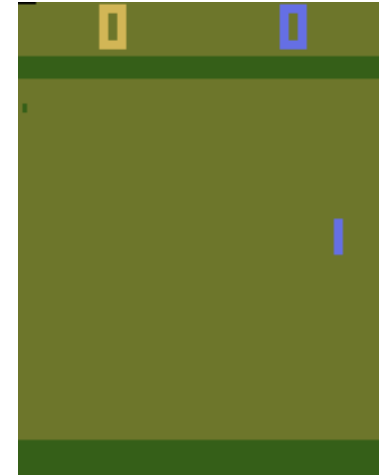
2. Apart from standard gradient descent, advanced (i.e., momentum-based) descent algorithms are possible (e.g., RMSProp, Adam, ...)

Example: Atari game Pong (1)

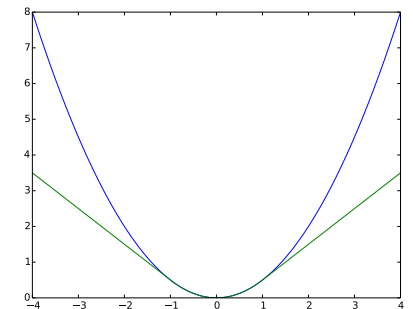
- *State s* :
 - 210×160 pixels with 3 channels each (RGB) and values $s_i \in \{0, \dots, 255\}$,
 - stacking of multiple frames,
 - some additional preprocessing.
- *Action a* : 18 actions in total, (6 buttons, some of which can be pushed at the same time).
- *Architecture for Q_θ* (Mnih et al. 2015):



- *Loss function $L(\theta)$* : Huber loss $L_\delta(\theta) = \begin{cases} \frac{1}{2}\theta^2, & \theta \leq \delta \\ \delta(|\theta| - \frac{1}{2}\theta), & \text{otherwise} \end{cases}$.
- *Training*: RMSProp descent algorithm.



Pong setup: More details in the [Farama Gymnasium](#) description.



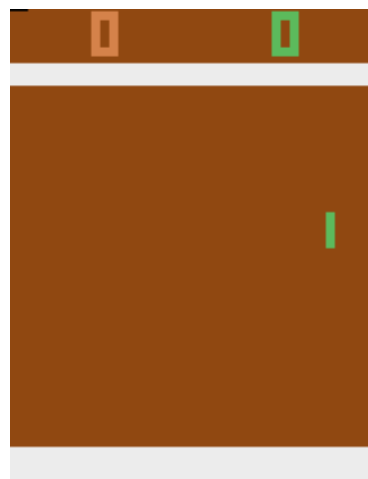
Huber loss ([Wikipedia](#))

- *Annealing* of the exploration rate: ϵ goes from 1 to 0.1 (or 0.05) over the first 10^6 frames.

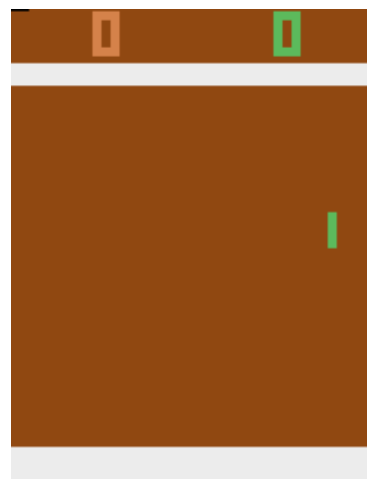
Example: Atari game Pong (2)



0 steps



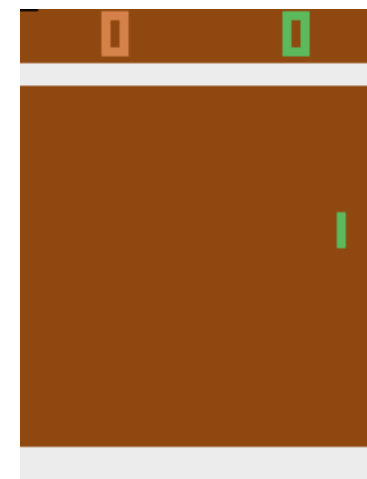
100,000 steps



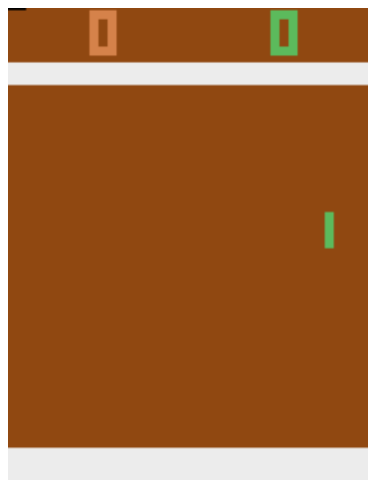
300,000 steps



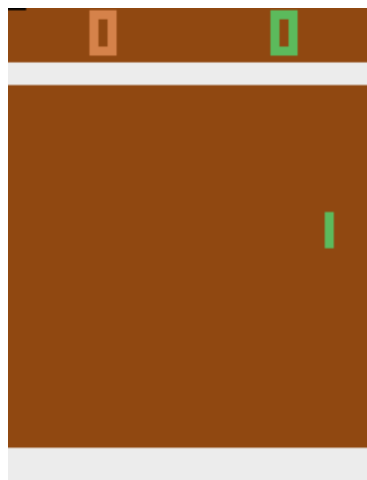
500,000 steps



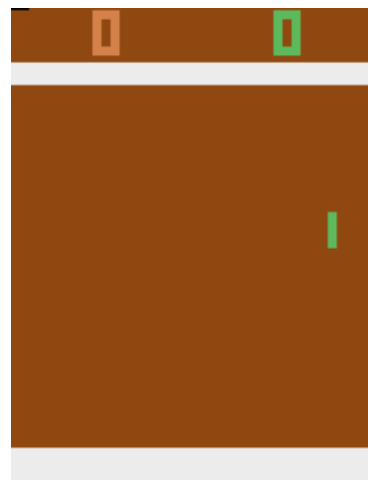
800,000 steps



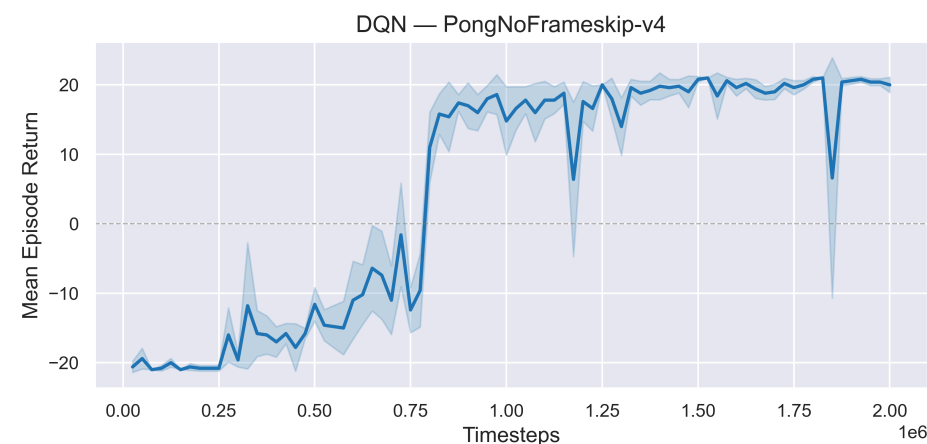
1,000,000 steps



1,500,000 steps

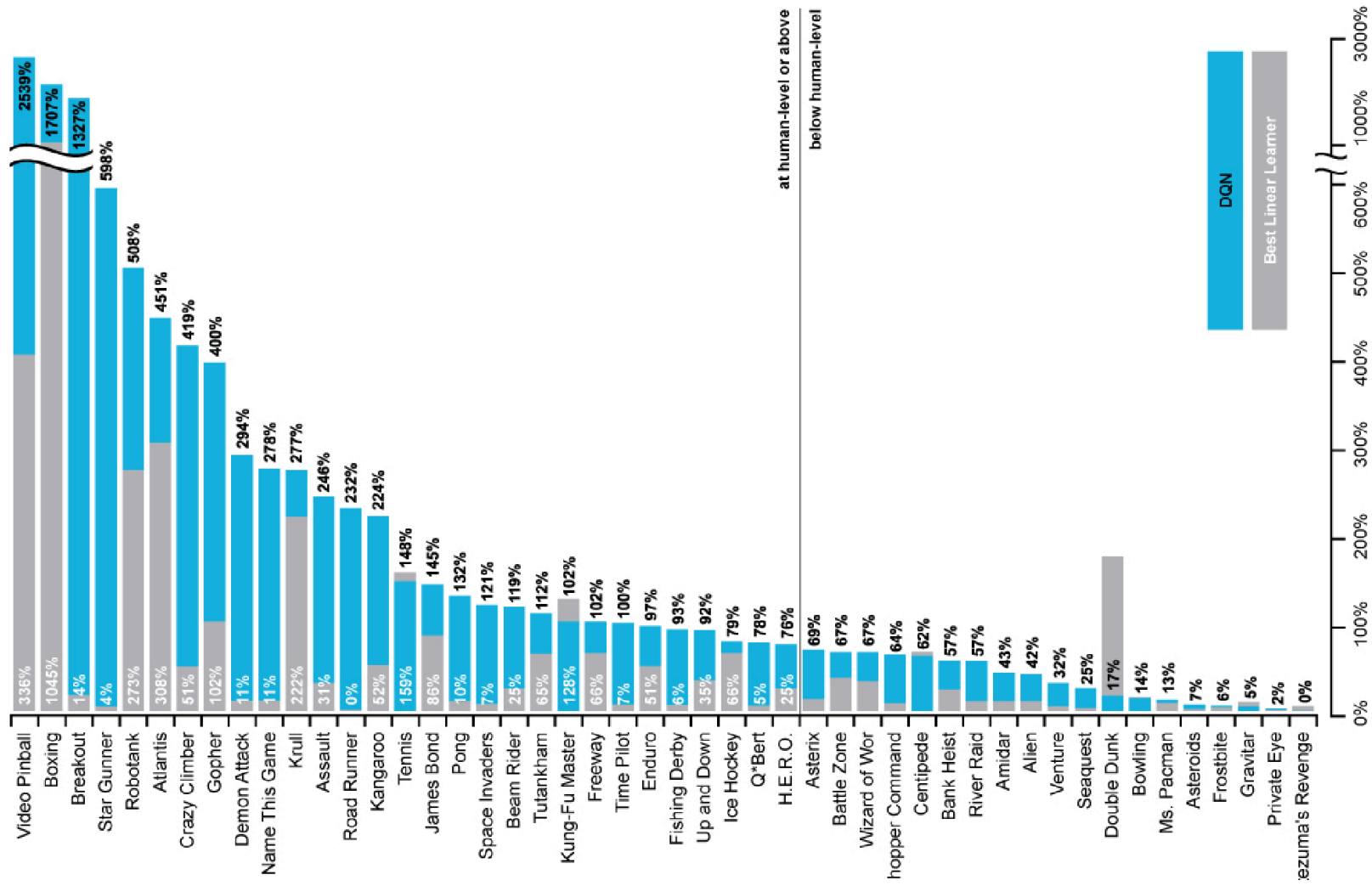


2,000,000 steps

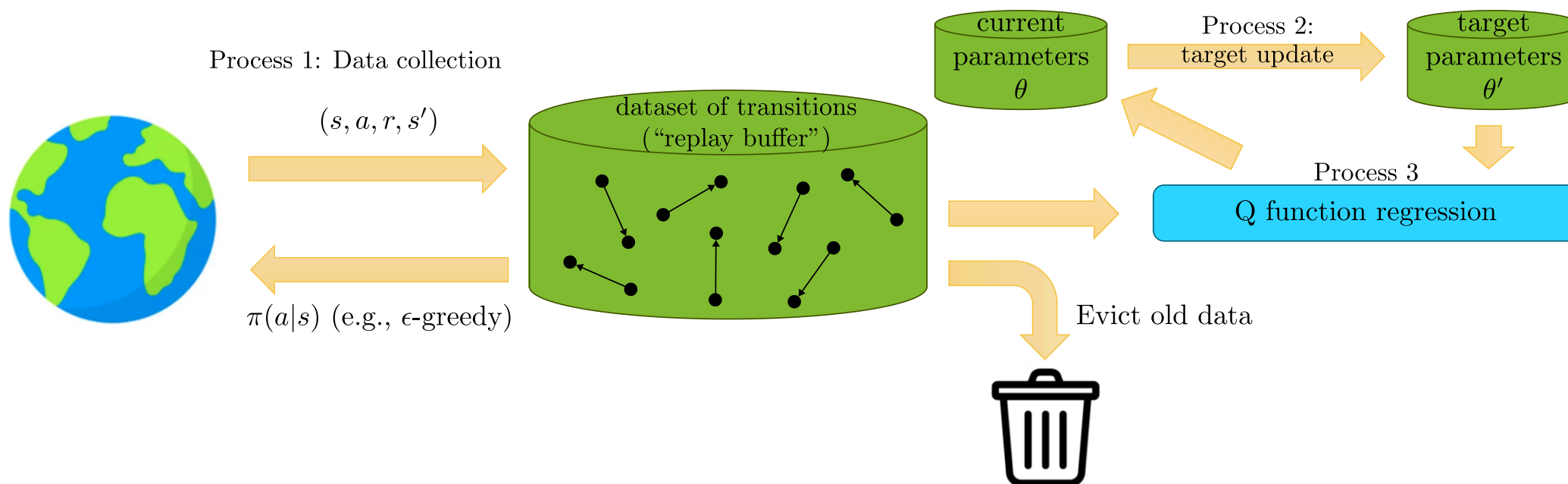


Return

Example: Atari games



The general view of Q learning



Inspired by the Sergey Levine's CS285 lecture.

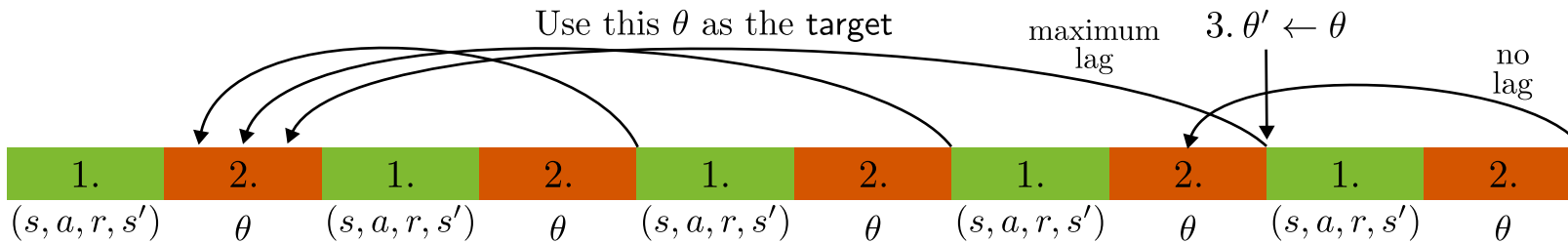
- **DQN**: Processes 1 and 2 run at the same speed, process 2 is slower.
- **Online Q learning**: evict immediately, Process 1, 2 and 3 all run at the same speed.
 - Value or Q learning as in the last lecture, followed directly by a policy improvement step.
 - Also similar to the value iteration algorithm from dynamic programming.

- Many variations: data collection and eviction strategies, frequencies and repetitions of the individual process steps, ...

DQN extensions & modifications

Alternative target networks (Polyak averaging)

- Consider the DQN algorithm and separate it into three main components:
 - Data collection, i.e., store tuples (s, a, r, s') in $\mathcal{D}$.
 - Improvement using mini batches $\mathcal{B}$ and a fixed target network, e.g., $\theta \leftarrow \theta + \alpha \sum_{i=1}^{|\mathcal{B}|} [y_i - Q_{\theta}(s_i, a_i)] \nabla_{\theta} Q_{\theta}(s_i, a_i)$.
 - Update target network every N_{θ} steps: $\theta' \leftarrow \theta$.
- Let's visualize these steps (with $N_{\theta} = 4$, even though it's usually much larger):



- The lag is different in different places! This doesn't need to be a problem, but it feels *uneven*.
- Alternative: **Polyak averaging**. In every step, perform the *linear interpolation*

$$\theta' \leftarrow \tau \theta' + (1 - \tau) \theta$$

- When initializing $\theta' = \theta$, this linear interpolation tends to work for nonlinear models (i.e., NNs) as well!

Double Q networks

Remember the *maximization bias* and double Q learning?

⇒ Don't use same network to choose the action and estimate the value!

Double deep Q learning:

- Just use the **current network** (θ) to evaluate the action.
- Still use the **target network** (θ') to evaluate the value.

$$y = r + \gamma Q_{\theta'}(s', \arg \max_a Q_{\theta}(s', a))$$

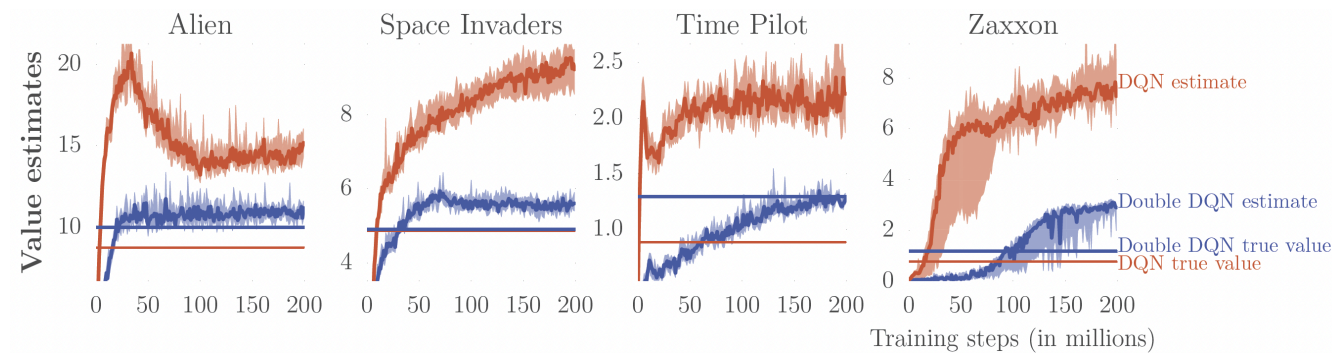
Algorithm: “Classical” Q-Learning.

- Take a_t (ϵ -greedy on $Q_1 + Q_2$), observe (r_t, s_{t+1})
- if $\text{rand}() > 0.5$ then

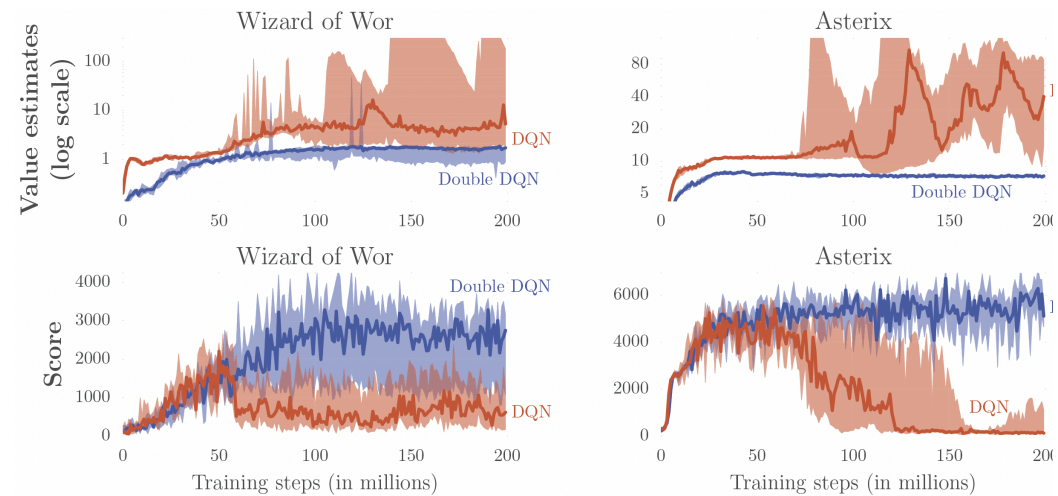
$$y = r_t + \gamma Q_2(s_{t+1}, \arg \max_a Q_1(s_{t+1}, a))$$

$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha [y - Q_1(s_t, a_t)]$$

- else: ...



(Hasselt, Guez, and Silver 2016)



(Hasselt, Guez, and Silver 2016)

Prioritized experience replay

n -step returns

Some remarks on continuous actions

Some practical tips

Least squares policy iteration

Linear action-value-function estimation

- Recall the LSTD algorithm from last lecture: batch least squares estimation $V^\pi(s)$ with linear models:

$$y = \begin{bmatrix} r_0 \\ \vdots \\ r_{T-1} \end{bmatrix}, \quad \Psi = \begin{bmatrix} \psi_0^\top - \gamma \psi_1^\top \\ \vdots \\ \psi_{T-1}^\top - \gamma \psi_T^\top \end{bmatrix}, \quad J(\theta) = \frac{1}{N} \|y - \Psi\theta\|_F^2, \quad \theta^* = (\Psi^\top \Psi)^{-1} \Psi^\top y = \Psi^\dagger y, \quad V_{\theta^*}(s) = \theta^{*\top} \psi(s).$$

- Let's do the same for Q values $\Rightarrow$ **LS-SARSA / LSTDQ**

$$Q_\theta(s, a) = \theta^\top \psi(s, a) = \sum_{i=1}^d \theta_i \psi_i(s, a) \quad \text{(Linear model)}$$

$$Q^\pi(s, a) \approx r + \gamma Q_\theta(s', a') \quad \text{(TD(0) bootstrapping)}$$

- Loss function:

$$J(\theta) = \sum_{t=0}^T \left(r_t - [\psi_t^\top - \gamma \psi_{t+1}^\top] \theta \right)^2 = \frac{1}{N} \|y - \Psi\theta\|_F^2, \quad \text{where } \psi_t = \psi(s_t, a_t).$$

- Same as before, only that the features depend on both states and actions: $\psi(s, a)$.

On-policy and off-policy LS-SARSA

- Same data collection procedure as before: batch mode $\Rightarrow \Psi$ and y .
- Regarding the data in Ψ , we can distinguish two cases:
 - the same policy π (*on-policy learning*),
 - an arbitrary policy $a' \sim \pi'(\cdot|s')$ (*off-policy learning*).
- If we apply off-policy LS-SARSA then
 - we retrieve the flexibility to collect training samples arbitrarily,
 - at the cost of an estimation bias based on the sampling distribution.
- Possible modifications:
 - To prevent numeric instability regularization is possible, e.g., ridge regression $\theta^* = (\Psi^\top \Psi + \epsilon I)^{-1} \Psi^\top y$
 - Recursive implementation for online usage is straightforward.

Least squares policy iteration (LSPI)

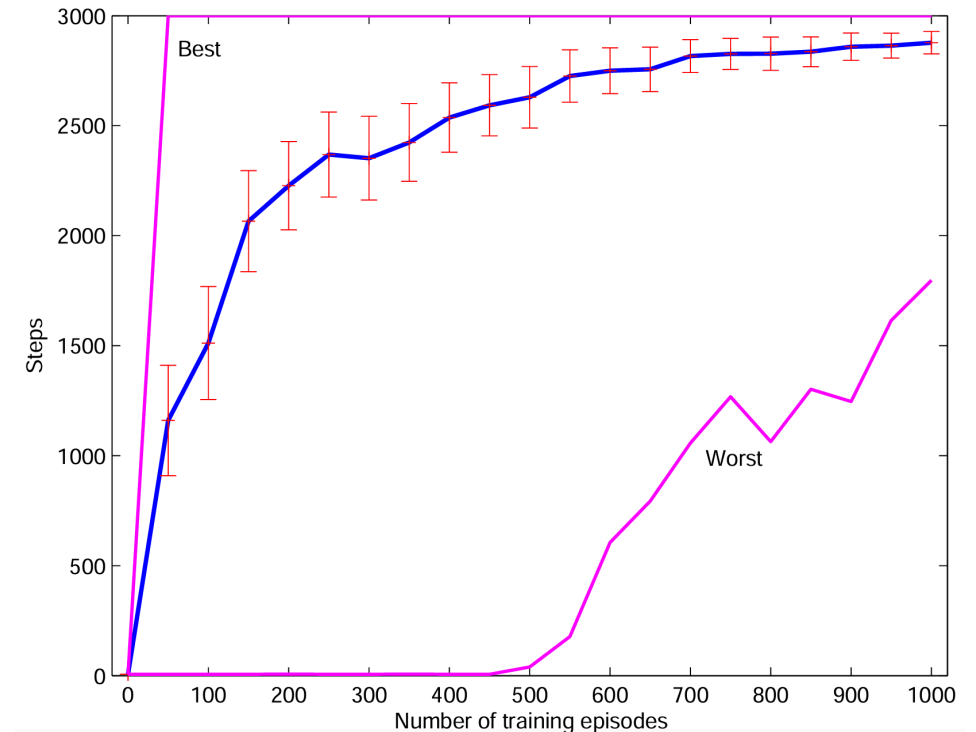
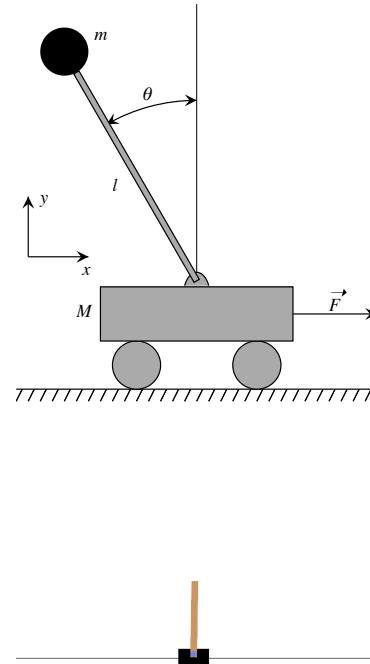
General idea:

- Apply general policy improvement (GPI) based on dataset $\mathcal{D}$,
- Policy evaluation by off-policy LS-SARSA,
- Policy improvement by greedy choices on predicted action values.

Some remarks:

- LSPI is an offline and off-policy control approach.
- Exploration is required by feeding suitable sampling distributions to $\mathcal{D}$:
 - ϵ -greedy choices based on Q_θ .
 - Completely random samples are conceivable as well.

Example: Pendulum on a cart ([Abdelwanis et al. 2026](#))



Balancing steps before episode termination with a clipping of maximum 3000 steps
([Lagoudakis and Parr 2003](#))

Summary / what you have learned

Summary / what you have learned

- The procedures from the approximate prediction can be transferred to value-based control in a straightforward way.
- Downside: the policy improvement theorem no longer applies in the approximate RL case.
 - Control algorithms may diverge.
 - Performance trade-offs between different parts of state-action space could emerge.
- Off-policy batch learning approaches allow for efficient data usage.
 - LSPI uses LS-SARSA on linear function approximation.
 - DQN extends Q-learning on non-linear approximation with additional tweaks (experience replay, target networks,...).
 - However, a prediction bias results (off-policy sampling distribution).

References

- Abdelwanis, Ali, Barnabas Haucke-Korber, Darius Jakobeit, Wilhelm Kirchgässner, Marvin Meyer, Maximilian Schenke, Hendrik Vater, Oliver Wallscheid, and Daniel Weber. 2026. "Reinforcement Learning: A Comprehensive Open-Source Course." *Journal of Open Source Education* 9 (97). The Open Journal: 306. doi:[10.21105/jose.00306](https://doi.org/10.21105/jose.00306).
- Hasselt, Hado van, Arthur Guez, and David Silver. 2016. "Deep Reinforcement Learning with Double q-Learning." *Proceedings of the AAAI Conference on Artificial Intelligence* 30 (1).
- Lagoudakis, Michail G, and Ronald Parr. 2003. "Least-Squares Policy Iteration." *Journal of Machine Learning Research* 4 (Dec): 1107–49.
- Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, et al. 2015. "Human-Level Control Through Deep Reinforcement Learning." *Nature* 518 (7540): 529–33.
- Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.